Document 05 — Backend Schema

Data Model & Auth Architecture for NowWise

Product Name

NowWise

Internal Platform

NowWise OS

Database Provider

Supabase Postgres

Auth Provider

Supabase Auth for admins
Telegram identity for end users

Document Purpose

This document defines how NowWise data is stored, related, secured, indexed, and accessed before writing migrations.

The goal is to avoid random schema decisions, painful migrations, and security holes.

---

1. Backend Architecture Summary

NowWise is a Telegram-first and voice-first AI lifestyle operating system.

The backend must store:

- Telegram users
- User goals and preferences
- Voice transcripts
- Food logs
- Product lookups
- Product database

- Product corrections
- Reminders
- Reminder actions
- Weekly reports
- Behavior patterns
- Admin review queue
- Privacy/consent settings
- File uploads
- Audit logs

Core schema principle:

> Every user interaction becomes a `user_event`.

Specific domain tables such as `food_logs`, `product_logs`, `voice_events`, and `reminder_events` should link back to `user_events`.

This gives NowWise one universal life timeline.

---

2. Database Extensions

Enable these Supabase/Postgres extensions.

```sql id="l9xmwk"
create extension if not exists "pgcrypto";
create extension if not exists "uuid-ossp";
create extension if not exists "pg_trgm";
create extension if not exists "vector";
```

```
| Extension | Purpose |
|---|---|
| `pgcrypto` | UUIDs, crypto utilities |
| `uuid-ossp` | UUID generation compatibility |
| `pg_trgm` | Fuzzy product search |
| `vector` | Embeddings for memory/product search |

---

# 3. Enum Strategy

Use Postgres enums for stable values.

## 3.1 user_role

```sql id="1mkr5w"
create type user_role as enum (
  'telegram_user',
```
```

```
  'super_admin',
  'admin',
  'reviewer',
  'product_admin',
  'support',
  'read_only'
);
```

3.2 event_type

```sql id="6ds9mt" create type event_type as enum ( 'food_log', 'product_lookup', 'voice_note', 'photo_upload', 'receipt_scan', 'restaurant_menu', 'reminder_created', 'reminder_checkin', 'water_log', 'daily_reflection', 'weekly_report', 'correction', 'privacy_request', 'unknown' );

```

3.3 event_source

```sql id="6xy1a4"
create type event_source as enum (
  'telegram_text',
  'telegram_voice',
  'telegram_photo',
  'telegram_document',
  'admin',
  'system',
  'api'
);
```

3.4 event_status

```sql id="lfiogs" create type event_status as enum ( 'received', 'processing', 'processed', 'needs_review', 'failed', 'deleted' );

```

3.5 reminder_action

```sql id="f8c7f9"
create type reminder_action as enum (
  'taken',
```

```
  'later',
  'skip',
  'no_response',
  'cancelled'
);
```

3.6 product_decision

```sql id="6mmw0d" create type product_decision as enum ( 'can_have', 'occasionally', 'avoid_today', 'ask_clarification', 'unknown' );
```

3.7 review_status

```sql id="n8wp9u"
create type review_status as enum (
  'pending',
  'approved',
  'corrected',
  'rejected',
  'ignored'
);
```

---

4. Core Identity Tables

4.1 Table: users

Stores all Telegram users and admin-linked user records.

```sql id="q5turk" create table users ( id uuid primary key default gen_random_uuid(),

telegram_user_id text unique, telegram_chat_id text, telegram_username text, telegram_first_name text, telegram_last_name text,

email text unique, full_name text,

role user_role not null default 'telegram_user',
```

timezone text not null default 'Asia/Kolkata', language_preference text not null default 'en', is_active boolean not null default true,

last_seen_at timestamptz, onboarded_at timestamptz,

created_at timestamptz not null default now(), updated_at timestamptz not null default now());

```
### Notes

- Telegram users may not have email.
- Admin users should have email.
- `telegram_user_id` is unique.
- `telegram_chat_id` is required for sending messages.
- `role` controls dashboard access.

### Indexes

```sql id="j65mus"
create index idx_users_telegram_user_id on users (telegram_user_id);
create index idx_users_telegram_chat_id on users (telegram_chat_id);
create index idx_users_email on users (email);
create index idx_users_role on users (role);
create index idx_users_last_seen_at on users (last_seen_at desc);
```

4.2 Table: user_profiles

Stores user goals, diet preferences, and safe health context.

```sql id="7f3rn8" create table user_profiles ( id uuid primary key default gen_random_uuid(), user_id uuid not null references users(id) on delete cascade,

primary_goal text, secondary_goals text[] default '{}',

diet_type text, allergies jsonb not null default '[]'::jsonb, disliked_foods jsonb not null default '[]'::jsonb, preferred_foods jsonb not null default '[]'::jsonb,

health_notes text, medical_disclaimer_acknowledged boolean not null default false,

quiet_hours_start time, quiet_hours_end time,

privacy_mode text not null default 'normal',

created_at timestamptz not null default now(), updated_at timestamptz not null default now(),

unique(user_id) );

```
Allowed Values

`primary_goal` examples:

```text id="akxu3n"
better_food_habits
digestion
sugar_control
weight_control
energy
hydration
custom
```

`diet_type` examples:

```text id="672moe" vegetarian non_vegetarian eggetarian vegan no_preference

```
Indexes

```sql id="qeoopb"
create index idx_user_profiles_user_id on user_profiles (user_id);
create index idx_user_profiles_primary_goal on user_profiles (primary_goal);
```

---

5. Universal Event System

5.1 Table: user_events

This is the main timeline table.

Every meaningful interaction must create a `user_event`.

```sql id="exq0ib" create table user_events ( id uuid primary key default gen_random_uuid(),

user_id uuid not null references users(id) on delete cascade,

event_type event_type not null default 'unknown', source event_source not null, status event_status not null default 'received',

raw_input text, ai_interpretation jsonb not null default '{}'::jsonb,

occurred_at timestamptz not null default now(), timezone text not null default 'Asia/Kolkata',

inferred_meal_type text, location_label text, location_data jsonb not null default '{}'::jsonb,

confidence_score numeric(4,3), error_message text,

parent_event_id uuid references user_events(id) on delete set null,

created_at timestamptz not null default now(), updated_at timestamptz not null default now() );

````
Important Rule

Use `occurred_at` as the real user event time.

Use `created_at` as database insert time.

If user says:

> I ate this yesterday

then `occurred_at` should represent yesterday, not current time after
confirmation.

Indexes

```sql id="19qqo9"
create index idx_user_events_user_id on user_events (user_id);
create index idx_user_events_user_time on user_events (user_id, occurred_at
desc);
create index idx_user_events_type on user_events (event_type);
create index idx_user_events_status on user_events (status);
create index idx_user_events_source on user_events (source);
create index idx_user_events_confidence on user_events (confidence_score);
create index idx_user_events_parent on user_events (parent_event_id);
````

---

6. Voice Schema

6.1 Table: voice_events

Stores metadata about Telegram voice notes and transcription.
````

```sql id="c4a79w" create table voice_events ( id uuid primary key default gen_random_uuid(),

user_id uuid not null references users(id) on delete cascade, event_id uuid references user_events(id) on delete cascade,

telegram_message_id text, telegram_file_id text,

audio_temp_path text, audio_mime_type text, audio_duration_seconds int, audio_size_bytes int,

provider_used text, provider_model text,

raw_transcript text, cleaned_transcript text, detected_language text,

confidence_score numeric(4,3), processing_time_ms int,

status event_status not null default 'received', error_message text,

raw_audio_deleted_at timestamptz,

created_at timestamptz not null default now(), updated_at timestamptz not null default now() );

````
Privacy Rule

Raw audio must be deleted after transcription by default.

`raw_audio_deleted_at` must be populated after cleanup.

Indexes

```sql id="ftkpmb"
create index idx_voice_events_user_id on voice_events (user_id);
create index idx_voice_events_event_id on voice_events (event_id);
create index idx_voice_events_status on voice_events (status);
create index idx_voice_events_provider on voice_events (provider_used);
create index idx_voice_events_created_at on voice_events (created_at desc);
create index idx_voice_events_confidence on voice_events (confidence_score);
````

6.2 Table: voice_provider_metrics

Tracks ASR provider performance.

```sql id="j1iu6f" create table voice_provider_metrics ( id uuid primary key default gen_random_uuid(),
```

provider_name text not null, provider_model text,

total_requests int not null default 0, success_count int not null default 0, failure_count int not null default 0,

avg_latency_ms numeric, avg_confidence numeric(4,3), avg_cost numeric(10,6),

period_start timestamptz not null, period_end timestamptz not null,

created_at timestamptz not null default now());

```
### Indexes

```sql id="pyb5uo"
create index idx_voice_provider_metrics_provider on voice_provider_metrics
(provider_name);
create index idx_voice_provider_metrics_period on voice_provider_metrics
(period_start, period_end);
```

---

7. Food Logging Schema

7.1 Table: food_logs

Stores structured meal/food data.

```sql id="485gtt" create table food_logs ( id uuid primary key default gen_random_uuid(),

user_id uuid not null references users(id) on delete cascade, event_id uuid not null references user_events(id) on delete cascade,

meal_type text, foods jsonb not null default '[]'::jsonb,

contains_sweet boolean not null default false, contains_fried boolean not null default false, contains_high_sodium boolean not null default false, contains_sugary_drink boolean not null default false,

protein_quality text, vegetable_presence boolean, fiber_quality text,

estimated_portion_text text, calorie_estimate numeric, macro_estimate jsonb not null default '{}'::jsonb,

ai_summary text, ai_recommendation text,

confidence_score numeric(4,3),

occurred_at timestamptz not null, created_at timestamptz not null default now(), updated_at timestamptz not null default now() );

```
Important Rule

Calories are optional and should not be the main product.

The product should focus on:

- pattern
- decision
- next action
- habit improvement

Indexes

```sql id="j3b07j"
create index idx_food_logs_user_id on food_logs (user_id);
create index idx_food_logs_event_id on food_logs (event_id);
create index idx_food_logs_user_time on food_logs (user_id, occurred_at desc);
create index idx_food_logs_meal_type on food_logs (meal_type);
create index idx_food_logs_contains_sweet on food_logs (contains_sweet);
create index idx_food_logs_contains_fried on food_logs (contains_fried);
```

7.2 Table: water_logs

Stores hydration events.

```sql id="g6udqw" create table water_logs ( id uuid primary key default gen_random_uuid(),

user_id uuid not null references users(id) on delete cascade, event_id uuid references user_events(id) on delete cascade,

amount_ml int, amount_text text,

occurred_at timestamptz not null, created_at timestamptz not null default now() );

```
Indexes

```sql id="4w14wk"
create index idx_water_logs_user_time on water_logs (user_id, occurred_at desc);
```

```

8. Product Intelligence Schema

8.1 Table: products

Main product database.

```sql id="k028zm" create table products ( id uuid primary key default gen_random_uuid(),

brand text, product_name text not null, normalized_name text not null,

category text, subcategory text,

barcode text, package_size text,

ingredients text, nutrition jsonb not null default '{}'::jsonb, allergens jsonb not null default '[]'::jsonb,

processing_level text, default_decision product_decision default 'unknown',

source text not null default 'manual', verification_status text not null default 'unverified',

image_url text, notes text,

created_by uuid references users(id) on delete set null, verified_by uuid references users(id) on delete set null, verified_at timestamptz,

created_at timestamptz not null default now(), updated_at timestamptz not null default now() );

```
Product Nutrition JSON Example

```json id="r4qof4"
{
  "serving_size": "30g",
  "calories": 180,
  "protein_g": 6,
  "sugar_g": 1,
  "sodium_mg": 220,
  "fiber_g": 3,
  "fat_g": 14
}
```

Indexes

```sql id="hzrvyo" create index idx_products_normalized_name on products using gin (normalized_name gin_trgm_ops); create index idx_products_brand on products (brand); create index idx_products_barcode on products (barcode); create index idx_products_category on products (category); create index idx_products_verification_status on products (verification_status);

```

8.2 Table: product_variants

Stores variants such as salted/unsalted/flavors/sizes.

```sql id="prfj6l"
create table product_variants (
  id uuid primary key default gen_random_uuid(),

  product_id uuid not null references products(id) on delete cascade,

  variant_name text not null,
  normalized_variant_name text not null,

  barcode text,
  package_size text,

  ingredients text,
  nutrition jsonb not null default '{}'::jsonb,
  allergens jsonb not null default '[]'::jsonb,

  is_default boolean not null default false,

  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);
```

Indexes

```sql id="0nxih6" create index idx_product_variants_product_id on product_variants (product_id); create index idx_product_variants_name on product_variants using gin (normalized_variant_name gin_trgm_ops); create index idx_product_variants_barcode on product_variants (barcode);

```

8.3 Table: product_logs

````
Stores user product lookups and AI decisions.

```sql id="8eptbe"
create table product_logs (
  id uuid primary key default gen_random_uuid(),

  user_id uuid not null references users(id) on delete cascade,
  event_id uuid not null references user_events(id) on delete cascade,

  product_id uuid references products(id) on delete set null,
  product_variant_id uuid references product_variants(id) on delete set null,

  query_text text not null,
  source event_source not null,

  decision product_decision not null default 'unknown',
  reason text,
  portion_guidance text,
  suggested_alternative text,

  matched_confidence numeric(4,3),
  recommendation_confidence numeric(4,3),

  occurred_at timestamptz not null,
  created_at timestamptz not null default now()
);
````

Indexes

```sql id="esxi4d" create index idx_product_logs_user_time on product_logs (user_id, occurred_at desc); create index idx_product_logs_product_id on product_logs (product_id); create index idx_product_logs_decision on product_logs (decision); create index idx_product_logs_confidence on product_logs (matched_confidence);

````

8.4 Table: product_corrections

Stores user/admin corrections.

```sql id="k81fe6"
create table product_corrections (
  id uuid primary key default gen_random_uuid(),

  product_log_id uuid references product_logs(id) on delete cascade,
  user_id uuid references users(id) on delete set null,
````
````

```
  correction_source text not null,
  original_product_id uuid references products(id) on delete set null,
  corrected_product_id uuid references products(id) on delete set null,

  original_text text,
  corrected_text text,
  correction_notes text,

  status review_status not null default 'pending',

  reviewed_by uuid references users(id) on delete set null,
  reviewed_at timestamptz,

  created_at timestamptz not null default now()
);
```

Indexes

```sql id="hvo8g4" create index idx_product_corrections_status on product_corrections (status); create index idx_product_corrections_product_log on product_corrections (product_log_id);
```
---

9. Restaurant and Grocery Schema

9.1 Table: restaurant_logs

Stores restaurant/menu interactions.

```sql id="iuo9q7"
create table restaurant_logs (
  id uuid primary key default gen_random_uuid(),

  user_id uuid not null references users(id) on delete cascade,
  event_id uuid not null references user_events(id) on delete cascade,

  restaurant_name text,
  menu_source text,
  menu_url text,
  menu_image_path text,

  extracted_items jsonb not null default '[]'::jsonb,
  recommended_items jsonb not null default '[]'::jsonb,
```

```
  avoid_items jsonb not null default '[]'::jsonb,

  ai_summary text,
  confidence_score numeric(4,3),

  occurred_at timestamptz not null,
  created_at timestamptz not null default now()
);
```

Indexes

```sql id="m4jxwi"
create index idx_restaurant_logs_user_time on restaurant_logs (user_id, occurred_at desc);
create index idx_restaurant_logs_restaurant_name on restaurant_logs (restaurant_name);
```

```
---

## 9.2 Table: grocery_receipts

Stores receipt OCR and grocery basket intelligence.

```sql id="za5rlb"
create table grocery_receipts (
  id uuid primary key default gen_random_uuid(),

  user_id uuid not null references users(id) on delete cascade,
  event_id uuid not null references user_events(id) on delete cascade,

  receipt_image_path text,
  store_name text,

  raw_ocr_text text,
  extracted_items jsonb not null default '[]'::jsonb,

  basket_summary text,
  missing_categories jsonb not null default '[]'::jsonb,
  recommendations jsonb not null default '[]'::jsonb,

  confidence_score numeric(4,3),

  occurred_at timestamptz not null,
  created_at timestamptz not null default now()
);
```
```

Indexes

```sql id="9ym51g" create index idx_grocery_receipts_user_time on grocery_receipts (user_id, occurred_at desc); create index idx_grocery_receipts_store_name on grocery_receipts (store_name);

```
---

10. Habit and Reminder Schema

10.1 Table: reminders

Stores recurring reminder configuration.

```sql id="1in4hu"
create table reminders (
  id uuid primary key default gen_random_uuid(),

  user_id uuid not null references users(id) on delete cascade,

  habit_name text not null,
  reminder_type text not null,

  frequency text not null,
  schedule_json jsonb not null default '{}'::jsonb,

  reminder_time time,
  timezone text not null default 'Asia/Kolkata',

  active boolean not null default true,

  created_from event_source not null,
  source_event_id uuid references user_events(id) on delete set null,

  last_sent_at timestamptz,
  next_run_at timestamptz,

  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);
```

Example `schedule_json`

```json id="1hbbr2" { "type": "daily", "time": "21:30", "timezone": "Asia/Kolkata", "days_of_week": [] }
```

```
### Indexes

```sql id="snsjeb"
create index idx_reminders_user_id on reminders (user_id);
create index idx_reminders_active_next_run on reminders (active, next_run_at);
create index idx_reminders_type on reminders (reminder_type);
```

10.2 Table: reminder_events

Stores every reminder occurrence.

```sql id="h5ucuf" create table reminder_events ( id uuid primary key default gen_random_uuid(),

reminder_id uuid not null references reminders(id) on delete cascade, user_id uuid not null references users(id) on delete cascade,

scheduled_at timestamptz not null, sent_at timestamptz,

user_action reminder_action not null default 'no_response', action_at timestamptz,

snooze_until timestamptz,

telegram_message_id text,

created_at timestamptz not null default now() );
```

```
### Indexes

```sql id="wt1ssb"
create index idx_reminder_events_user_time on reminder_events (user_id,
scheduled_at desc);
create index idx_reminder_events_reminder_id on reminder_events (reminder_id);
create index idx_reminder_events_action on reminder_events (user_action);
create index idx_reminder_events_snooze_until on reminder_events (snooze_until);
```

---

11. Behavior Intelligence Schema

11.1 Table: behavior_patterns

Stores weekly/monthly patterns.

```sql id="h14tm9" create table behavior_patterns ( id uuid primary key default gen_random_uuid(),

user_id uuid not null references users(id) on delete cascade,

pattern_type text not null, period_start date not null, period_end date not null,

evidence jsonb not null default '[]'::jsonb, insight text not null,

severity text not null default 'low', confidence_score numeric(4,3),

shown_to_user boolean not null default false, shown_at timestamptz,

created_at timestamptz not null default now() );

```
Pattern Types

```text id="qv1x76"
sweets_high_frequency
high_sodium_snacks
late_night_eating
low_protein
low_vegetables
low_hydration
habit_skipping
restaurant_frequency
```

Indexes

```sql id="dmmy9p" create index idx_behavior_patterns_user_period on behavior_patterns (user_id, period_start desc, period_end desc); create index idx_behavior_patterns_type on behavior_patterns (pattern_type); create index idx_behavior_patterns_severity on behavior_patterns (severity);
```

```
---

## 11.2 Table: weekly_reports

Stores weekly user reports.

```sql id="aem4tx"
create table weekly_reports (
```
```

```
  id uuid primary key default gen_random_uuid(),

  user_id uuid not null references users(id) on delete cascade,

  week_start date not null,
  week_end date not null,

  food_summary text,
  product_summary text,
  habit_summary text,
  pattern_summary text,
  best_improvement text,
  next_week_focus text,

  full_report_json jsonb not null default '{}'::jsonb,

  status text not null default 'draft',
  sent_at timestamptz,
  telegram_message_id text,

  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now(),

  unique(user_id, week_start, week_end)
);
```

Indexes

```sql id="r9h06n"
create index idx_weekly_reports_user_week on weekly_reports (user_id, week_start desc);
create index idx_weekly_reports_status on weekly_reports (status);
```

```
---

# 12. Admin Review Schema

---

## 12.1 Table: admin_reviews

Universal review queue.

```sql id="a3pz9p"
create table admin_reviews (
  id uuid primary key default gen_random_uuid(),

  user_id uuid references users(id) on delete set null,
```
```

```
  event_id uuid references user_events(id) on delete cascade,

  review_type text not null,

  original_input text,
  ai_output jsonb not null default '{}'::jsonb,
  corrected_output jsonb,

  confidence_score numeric(4,3),
  status review_status not null default 'pending',

  assigned_to uuid references users(id) on delete set null,
  reviewed_by uuid references users(id) on delete set null,
  reviewed_at timestamptz,

  admin_notes text,

  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);
```

Review Types

```text id="0yjuef"
voice_transcript
food_parse
product_match
reminder_parse
restaurant_menu
receipt_ocr
weekly_report
privacy_request
```

```
### Indexes

```sql id="1nmja7"
create index idx_admin_reviews_status on admin_reviews (status);
create index idx_admin_reviews_type on admin_reviews (review_type);
create index idx_admin_reviews_confidence on admin_reviews (confidence_score);
create index idx_admin_reviews_created_at on admin_reviews (created_at desc);
create index idx_admin_reviews_assigned_to on admin_reviews (assigned_to);
```

---

13. Privacy and Consent Schema

13.1 Table: consent_settings

Stores user data permissions.
```

```sql id="c8jg57" create table consent_settings ( id uuid primary key default gen_random_uuid(),

user_id uuid not null references users(id) on delete cascade,

store_transcripts boolean not null default true, store_raw_audio boolean not null default false, store_photos boolean not null default false,

allow_product_personalization boolean not null default true, allow_behavior_insights boolean not null default true, allow_aggregated_analytics boolean not null default false, allow_research boolean not null default false, allow_partner_sharing boolean not null default false,

consent_version text not null default 'v1',

created_at timestamptz not null default now(), updated_at timestamptz not null default now(),

unique(user_id) );

```
Important Rule

`allow_partner_sharing` must default to false.

No insurance/wellness partner data sharing without explicit opt-in.

13.2 Table: privacy_requests

Stores data export/delete requests.

```sql id="re8hfs"
create table privacy_requests (
  id uuid primary key default gen_random_uuid(),

  user_id uuid not null references users(id) on delete cascade,

  request_type text not null,
  status text not null default 'pending',

  requested_at timestamptz not null default now(),
  completed_at timestamptz,

  requested_from event_source not null default 'telegram_text',

  admin_notes text,
```

```
  handled_by uuid references users(id) on delete set null
);
```

Request Types

```text id="f9rg4x"
export_data
delete_data
delete_account
change_consent
```

```
### Indexes

```sql id="pshffd"
create index idx_privacy_requests_user_id on privacy_requests (user_id);
create index idx_privacy_requests_status on privacy_requests (status);
create index idx_privacy_requests_type on privacy_requests (request_type);
```

---

14. Audit Logging

14.1 Table: audit_logs

Stores sensitive admin/system actions.

```sql id="2se6na"
create table audit_logs (
  id uuid primary key default gen_random_uuid(),

  actor_user_id uuid references users(id) on delete set null,
  actor_role user_role,

  action text not null,
  entity_type text not null,
  entity_id uuid,

  before_data jsonb,
  after_data jsonb,

  ip_address text,
  user_agent text,

  created_at timestamptz not null default now()
);
```

```
### Actions to Audit

```text id="7iaxzl"
admin_login
product_created
product_updated
product_merged
review_approved
```

```
review_corrected
user_data_exported
user_data_deleted
consent_updated
reminder_deleted
weekly_report_sent
```

Indexes

```sql id="y03y9k"
create index idx_audit_logs_actor on audit_logs (actor_user_id);
create index idx_audit_logs_entity on audit_logs (entity_type, entity_id);
create index idx_audit_logs_created_at on audit_logs (created_at desc);
```

```
---

# 15. File and Media Storage

Use Supabase Storage.

---

## 15.1 Buckets

| Bucket | Purpose | Public? |
|---|---|---|
| `voice-temp` | Temporary raw audio | No |
| `food-photos` | Food/product photos | No |
| `receipt-images` | Grocery receipts | No |
| `menu-images` | Restaurant menus | No |
| `product-images` | Product images | No or controlled |
| `exports` | User data exports | No |

---

## 15.2 Storage Paths

```text id="iytk1h"
voice-temp/{user_id}/{voice_event_id}.ogg
food-photos/{user_id}/{event_id}.jpg
receipt-images/{user_id}/{receipt_id}.jpg
menu-images/{user_id}/{restaurant_log_id}.jpg
product-images/{product_id}/{image_id}.jpg
exports/{user_id}/{privacy_request_id}.json
```

```

15.3 Storage Rules

Rule	Requirement
Raw voice audio	Delete after transcription
Food photos	Store only if needed
Receipt images	Store only if needed
Exports	Expire after 7 days
Admin access	Role-based
Public access	Disabled by default

---

16. Relationships

16.1 Main Relationships

```text id="g84r12"
users 1 → 1 user_profiles
users 1 → many user_events
users 1 → many voice_events
users 1 → many food_logs
users 1 → many product_logs
users 1 → many reminders
users 1 → many reminder_events
users 1 → many behavior_patterns
users 1 → many weekly_reports
users 1 → many privacy_requests

user_events 1 → 0/1 voice_events
user_events 1 → 0/1 food_logs
user_events 1 → 0/1 product_logs
user_events 1 → 0/1 restaurant_logs
user_events 1 → 0/1 grocery_receipts

products 1 → many product_variants
products 1 → many product_logs
product_logs 1 → many product_corrections

reminders 1 → many reminder_events
```

```
---

# 17. Auth Model

---

## 17.1 End User Auth

Telegram users are identified by:

```text id="0Oxj8q"
```
```

```
telegram_user_id
telegram_chat_id
```

MVP does not require email login for normal users.

17.2 Admin Auth

Admins authenticate using Supabase Auth.

Admin role is stored in `users.role`.

17.3 Roles

Role	Permissions
`telegram_user`	Own Telegram data only
`reviewer`	Review low-confidence events
`product_admin`	Manage product database
`support`	View user issues, limited sensitive data
`admin`	Most admin dashboard access
`super_admin`	Full access, user deletion, role changes
`read_only`	Read dashboard only

---

18. Row Level Security

Enable RLS on all sensitive tables.

```sql id="sdresl" alter table users enable row level security; alter table user_profiles enable row level security; alter table user_events enable row level security; alter table voice_events enable row level security; alter table food_logs enable row level security; alter table product_logs enable row level security; alter table reminders enable row level security; alter table reminder_events enable row level security; alter table behavior_patterns enable row level security; alter table weekly_reports enable row level security; alter table consent_settings enable row level security; alter table privacy_requests enable row level security;

```

```
```

```
## 18.1 Helper Function: current_app_user_id

For authenticated web users:

```sql id="klyao0"
create or replace function current_app_user_id()
returns uuid
language sql
stable
as $$
  select id from users where email = auth.jwt() ->> 'email' limit 1;
$$;
```

18.2 Helper Function: is_admin

```sql id="6dqopb" create or replace function is_admin() returns boolean language sql stable as $$ select exists ( select 1 from users where email = auth.jwt() ->> 'email' and role in ('super_admin', 'admin', 'reviewer', 'product_admin', 'support', 'read_only') ); $$;

```

18.3 Helper Function: has_role

```sql id="gl0xdc"
create or replace function has_role(required_roles user_role[])
returns boolean
language sql
stable
as $$
  select exists (
    select 1
    from users
    where email = auth.jwt() ->> 'email'
      and role = any(required_roles)
  );
$$;
```

18.4 Example RLS: Users Can Read Own Profile

```sql id="uxpcms" create policy "Users can read own profile" on user_profiles for select using (user_id = current_app_user_id());

```

18.5 Example RLS: Admins Can Read Profiles

```sql id="of5yuc"
create policy "Admins can read profiles"
on user_profiles
for select
using (is_admin());
```

18.6 Example RLS: Admin Product Access

```sql id="od0z28" create policy "Product admins can manage products" on products for all using (has_role(array['super_admin','admin','product_admin']::user_role[])) with check (has_role(array['super_admin','admin','product_admin']::user_role[]));

```

18.7 Important Server-Side Rule

Telegram bot and workers should use Supabase service role key **only on the server**.

Never expose:

```text id="y9q2uf"
SUPABASE_SERVICE_ROLE_KEY
```

to frontend/browser.

---

19. Sensitive Fields

19.1 Sensitive Data

Field/Data	Sensitivity	Handling
Voice audio	High	Temporary, delete by default
Voice transcript	High	Store only if consent allows
Food logs	High	User-owned, RLS protected
```

Field/Data	Sensitivity	Handling
Health notes	High	Avoid diagnosis, RLS protected
Photos/receipts	High	Private buckets
Consent settings	High	Audit changes
Privacy requests	High	Admin-only
Telegram IDs	Medium	Protected
Admin actions	High	Audit logged

19.2 Encryption Recommendation

Supabase already encrypts at rest at infrastructure level.

For extra-sensitive future fields, use application-level encryption.

Potential future encrypted fields:

```text id="48v8ff" health_notes raw_transcript export_files location_data

```
Do not store payment method data directly. Use Stripe later.

---

20. Webhooks and Triggers

20.1 Updated At Trigger

Create reusable trigger.

```sql id="2ihr1q"
create or replace function set_updated_at()
returns trigger
language plpgsql
as $$
begin
  new.updated_at = now();
  return new;
```

```
end;
$$;
```

Apply to tables with `updated_at`.

```sql id="fslhur" create trigger set_users_updated_at before update on users for each row execute function set_updated_at();
```

```
Repeat for:

```text id="o16dbo"
user_profiles
user_events
voice_events
food_logs
products
product_variants
reminders
weekly_reports
admin_reviews
consent_settings
```

20.2 Auto Review Trigger

If confidence is low, create admin review.

Pseudo-rule:

```text id="jyia68" if confidence_score < 0.70 then create admin_reviews row
```

```
This can be done in application logic first.

Do not overuse database triggers for AI review logic in MVP.

---

# 21. API Endpoint List

---

## 21.1 Telegram
```
```

```text id="chlfss"
POST /api/telegram/webhook
```

Receives Telegram updates.

21.2 Voice

```text id="wyptpq" POST /api/voice/process POST /v1/audio/transcriptions

```
Processes Telegram voice notes and local ASR.

21.3 Events

```text id="yxauqi"
GET /api/events
GET /api/events/:id
PATCH /api/events/:id
POST /api/events/:id/correct
```

21.4 Food Logs

```text id="hav5yj" GET /api/food-logs POST /api/food-logs PATCH /api/food-logs/:id DELETE /api/food-logs/:id

```

21.5 Products

```text id="gojewr"
GET /api/products/search
GET /api/products/:id
POST /api/products
PATCH /api/products/:id
POST /api/products/:id/verify
POST /api/products/merge
```

21.6 Reminders

```text id="wgngxz" GET /api/reminders POST /api/reminders PATCH /api/reminders/:id DELETE /api/reminders/:id POST /api/reminders/:id/checkin

```

21.7 Reports

```text id="h27v1w"
POST /api/reports/weekly/generate
GET /api/reports/weekly/:user_id
POST /api/reports/weekly/:id/send
```

21.8 Admin Reviews

```text id="xu374v" GET /api/admin/reviews GET /api/admin/reviews/:id POST /api/admin/reviews/:id/approve POST /api/admin/reviews/:id/correct POST /api/admin/reviews/:id/reject

```

21.9 Privacy

```text id="sh5i5p"
GET /api/privacy/settings
PATCH /api/privacy/settings
POST /api/privacy/export
POST /api/privacy/delete-request
```

---

22. Migration Order

Build migrations in this order.

```text id="xdvfdh" 001_enable_extensions.sql 002_create_enums.sql 003_create_users_and_profiles.sql 004_create_user_events.sql 005_create_voice_events.sql 006_create_food_and_water_logs.sql 007_create_products_and_variants.sql 008_create_product_logs_and_corrections.sql 009_create_restaurant_and_receipt_logs.sql 010_create_reminders.sql 011_create_behavior_patterns_and_reports.sql 012_create_admin_reviews.sql 013_create_consent_privacy_audit.sql 014_create_indexes.sql 015_enable_rls.sql 016_create_storage_buckets.sql 017_create_functions_and_triggers.sql
```

````
---

# 23. MVP Schema Checklist

Build these first:

| Table | MVP? |
|---|---|
| users | Yes |
| user_profiles | Yes |
| user_events | Yes |
| voice_events | Yes |
| food_logs | Yes |
| products | Yes |
| product_variants | Yes |
| product_logs | Yes |
| reminders | Yes |
| reminder_events | Yes |
| behavior_patterns | Yes |
| weekly_reports | Yes |
| admin_reviews | Yes |
| consent_settings | Yes |
| privacy_requests | Yes |
| audit_logs | Yes |
| restaurant_logs | Basic |
| grocery_receipts | Basic |
| water_logs | Basic |

---

# 24. Final Backend Schema Decision

The correct NowWise backend architecture is:

```text id="jao5o0"
users
→ user_profiles
→ user_events as universal timeline
→ voice_events / food_logs / product_logs / reminders
→ behavior_patterns
→ weekly_reports
→ admin_reviews
→ consent_settings and privacy_requests
````

The schema must protect the central product promise:
````

> The user speaks or types naturally. NowWise captures the event, understands it, remembers it, recommends the next action, and protects the user's trust.

Therefore:

- Use `user_events` as the central event timeline.
- Use domain tables for structured intelligence.
- Use Supabase RLS for access control.
- Use admin review queues for AI uncertainty.
- Use private storage buckets for media.
- Delete raw audio by default.
- Never design the schema around selling personal health data.